Python

Dag 1 Printen in de console van Pycharm
Printen, inputs, commentaar, debuggeng, Name errors,
Syntax errors, String manipulatie, Variabelen

1) print ("Hello world!") een stuk tekst = "een string"

als er een rode streep onder de tekst staat = een error
de syntax error vertelt ons wat de fout is bij het uitvoeren van de code

String manipulation:
2) print ("Hello world! \n Hello world! \n Hello world") =
Hello world!
Hello world!
Hello world!

print ("Hello" + "Bart") = HelloBart
print ("Hello " + "Bart") =
print ("Hello" + " Bart") =
print ("Hello" + " " + "Bart") = } Hello Bart

Opletten met spaties voor de syntax, is gevoelig voor inspringen
voorkeur voor 4 spaties IPV TAB toets

Inputs:
input ("What is your name?") : What is your name? | -> cursor
De gebruiker kan nu iets ingeven
print ("Hello" + input ("What is your name?")) =
What is your name? | Bart -> enter : Hello Bart

print ("Hello " + input ("What is your name?") + "!") =
Hello Bart!

② Python

Variables: variabelen zijn namen waar je een waarde kan in opslaan, een beetje te vergelijken met een lege doos

```
name = input("What's your name?")
print(name)
```

of

```
name = "Bart"
print(name)
```

onze doos name krijgt de waard Bart
printen van de waarde Bart uit doos name

```
name = "Sofie"
print name
```

we kunnen de waarde in de doos steeds wijzigen
printen van de waarde Sofie uit de doos name

We kunnen met de functie len de lengte van de waarde meten

```
print(len(input("What is your name?")))
```

Bart
dit zal de waarde 4 printen

```
username = input("What is your name?")
length = len(username)
print(length)
```

Je mag namen kiezen voor je variabelen maar hou het leesbaar.
gebruik geen namen die python al gebruikt

Teach day 1:

```
print("Welcome to the Band name generator")
city = input("Which city did you grow up in?\n")
pet = input("What is the name of a pet?\n")
print("Your Band name could be: " + city + " " + pet)
```

3

Python

Data types, Numbers, Operations, Type conversions, f-string

Dag 2

Data types:

Strings = stukken tekst "What is your name?" of "Hello"
print ("Hello" [0]) = H → is Subscripting

Integer = volledig geheel getal
print (123 + 345) = 468

float = floating Point Number
print (3,14159)

Boolean = True or fals
print (~~Fals~~ True)
print (False)

Type checking:
print (len ("12345")) = 5
print (type ("ABC")) = str van string
print (type (5678)) = int van integer
print (type (3,14159)) = float
print (type (True)) = bool van boolean

Type conversion:
print (int ("123") + int ("456")) zal ervoor zorgen dat de strings worden omgezet naar integer en de berekening zal uitvoeren = 579

Mathemstical operations:
print ("My age: " + str (12)) = My age: 12
print (123 + 456) = 579
print (7 - 3) = 4

print (3 * 3 + 3 / 3 - 3) = 7.0
print (3 * (3 + 3) / 3 - 3) = 3.0

print (3 * 2) = 6
print (5 / 3) = 1.666666
print (5 // 3) = 1
print (2 ** 3) = 8

(4)

Python
Number manipulation:

```
bmi = 84 / 1.65 ** 2
print(bmi) = 30.853994490358113
print(int(bmi)) = 30
print(round(bmi)) = 31
print(round(bmi,2)) = 30.85
```

f string

```
age = 12
print(f"I am {age} years old")
```

Dag 3

If Else, Modulo, Nesting and elif, Mutiple if's

```
if else:
print("Welcome to the rollercoaster")
height = int(input("What is your height in cm?"))

if height >= 120:
    print("You can ride the rollercoaster")
else:
    print("Sorry you have to grow taller before you can ride.")
```

Modulo:

```
print(10 %3) = hoeveel keer gaat 3 in 10 = 3 keer met rest (1)
print("geef een getal in, ik zal nakijken of het even of oneven is")
getal = int(input("geef een geheel getal in"))

if (getal % 2) == 0:
    print("Het getal is even")
else:
    print("Het getal is oneven")
```

(5)

Python
Nesting and Elif:

```
print("Welcome to the rollercoaster!")
height = int(input("What is your height in cm?"))

if height >= 120:
    print("you can ride the rollercoaster")
    age = int(input("Whats your age"))
    if age <= 12:
        print("You have to pay €5")
    elif age <= 18:
        print("You have to pay €7")
    elif age <= 21:
        print("You have to pay €12")
    else:
        print("You are an adult you pay €15")
else:
    print("Sorry you have to grow taller before you can ride.")
```

Multiple if's : je kan meerdere ifs ook nog gebruiken

```
temperatuur = int(input("Hoeveel graden is het buiten?"))
if temperatuur > 30:
    print("Het is echt warm vandaag")
if temperatuur > 20:
    print("Het is aangenaam buiten")
if temperatuur > 10:
    print("Het is fris, maar nog OK")
if temperatuur < 10:
    print("Doe het een jas aan")
```

Logical operators
and = beide waar
or = 1 waar
not = het omgekeerde

Python
Random module, Lists

Dag 4

Om random te kunnen gebruiken moet je deze importeren

- Importeren random module

import random -> staat bv op lijn 1, kijk op de python doc's voor de verschillende type random

bv: random_integer = random.randint(1, 10]

Dit zal een willekeurig getal weergeven tussen 1 en 10
1 en 10 zijn er ook bij dus inclusief 1 en 10

Lists:

IPV 1 waarde in een variabele te plaatsen, gaan we nu meerdere waarden in een variabele plaatsen

Syntax:

fruits = [item 1, item 2]

fruits = ["Appel", "Peer", "kers", "banaan"]

in een lyst kunnen verschillende datatypes zitten

person = ["Bart", "Brondeel", 48, 1.78, "Borsleke"]

item toevoegen -> person.append("0494/66.69.70")
uitbreiden (itereerbaar) -> person.extend("MAN") = ["M", "A", "N"]
verwijderen (1ste item) -> person.remove("Bart")
wissen (alle items) -> person.clear()
index, geeft my de plaats van een item terug -> person.index(48) = 2
Sorteren be alfabetisch rangschikken -> person.sort() ->
zal niet werken met verschillende datatypes -> fruits.sort() wel

(6)$_2$ Day 4


```
import random
rock = '''
    _______
---'   ____)
      (_____)
      (_____)
      (____)
---.__(___)
'''

paper = '''
    _______
---'   ____)____
          ______)
          _______)
         _______)
---.__________)
'''

scissors = '''
    _______
---'   ____)____
          ______)
       __________)
      (____)
---.__(___)
'''

game_images = [rock, paper, scissors]
computer_choice = random.randint(0,2)
player_choice = int(input("what do you choose? Type 0 for rock, 1 for paper or 2 for scissors?"))

if player_choice == 0:
    print(f"Player chose: \n", rock)
elif player_choice == 1:
    print(f"Player chose: \n", paper)
elif player_choice == 2:
    print(f"Player chose: \n", scissors)

if computer_choice == 0:
```

(7)

Python
for loops, for loops with range

For loop: Syntax voorbeeld

```
fruits = ["Apple", "Peach", "Pear"]
for fruit in fruits:
    print (fruit)
    print (fruit + "PIE")
print (fruits)
```

Apple
Apple PIE
Peach
Peach PIE
Pear
Pear PIE
['Apple', 'Peach', 'Pear']

For loop with range:

```
for i in range (1, 101) # van 1 tot en met 100
    if i % 3 == 0 and i % 5 == 0:
        print ("EisBuss")
    elif i % 3 == 0:
        print ("Eiss")
    elif i % 5 == 0:
        print ("Buss")
    else:
        print (i)
```

(7)₂ Day 5


```python
import random

letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
numbers = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
symbols = ['!', '#', '$', '%', '&', '(', ')', '*', '+']

print("Welcome to the PyPassword Generator!")
nr_letters = int(input("How many letters would you like in your password?\n"))
nr_symbols = int(input(f"How many symbols would you like?\n"))
nr_numbers = int(input(f"How many numbers would you like?\n"))

#easy level

# paswoord = ""
# for char in range(0, nr_letters):
#     paswoord += random.choice(letters)
#
# for char in range(0, nr_numbers):
#     paswoord += random.choice(numbers)
#
# for char in range(0, nr_symbols):
#     paswoord += random.choice(symbols)
# print(paswoord)

paswoord_list = []
for char in range(0, nr_letters):
    paswoord_list.append(random.choice(letters))

for char in range(0, nr_numbers):
    paswoord_list.append(random.choice(numbers))

```

```
for char in range(0, nr_symbols):
    paswoord_list.append(random.choice(symbols))

print(paswoord_list)
random.shuffle(paswoord_list)
print(paswoord_list)

paswoord = ''
for char in paswoord_list:
    paswoord += char
print(f"Your password is: {paswoord}")
```

(7) 3 Day 5


```
1  student_scores = [150, 142, 185, 120, 171, 184, 149,
   24, 59, 68, 199, 78, 65, 89, 86, 55, 91, 64, 89]
2  #print(range(1, 10))
3
4  total_score = sum(student_scores)
5  print(total_score)
6
7  som = 0
8  for score in student_scores:
9      som += score
10
11 print(som)
12
13 max_score = 0
14 for highest_score in student_scores:
15     if highest_score > max_score:
16         max_score = highest_score
17 print(max_score)
18
19 min_score = max_score
20 for lowest_score in student_scores:
21     if lowest_score < min_score:
22         min_score = lowest_score
23 print((min_score))
```

(8)

Python
Functions, Code Blocks, While loops

Dag 6

Python heeft al ingebouwde functies zoals:
print() len() int() str() enz... zie ook de Python docs

Onze eigen functies maken doen we als volgt
def my_function():

bv: def my_function():
    print("Hello")
    print("Bart")

my_function() → calling the function → anders wordt de code niet uitgevoerd

oefeningen gemaakt in Replit world

While loop: de code blijft lopen zolang niet voldaan is aan de voorwaarde

```
aantal_beer = 5
while aantal_beer > 0:
    aantal_beer -= 1
    print(aantal_beer)
```

Dit zal het volgende printen: 4, 3, 2, 1, 0

9 Day 7

File - C:\Users\Bart Sofie\PycharmProjects\100 Days of Code - The Complete Python Pro Bootcamp\Day 7\task\main.py

```
import random

# TODO-1: - Update the word list to use the 'word_list' from hangman_words.py
from hangman_words import word_list
from hangman_art import  stages, logo

lives = 6

# TODO-3: - Import the logo from hangman_art.py and print it at the start of the game.
print(logo)

chosen_word = random.choice(word_list)
print(chosen_word)

placeholder = ""
word_length = len(chosen_word)
for position in range(word_length):
    placeholder += "_"
print("Word to guess: " + placeholder)

game_over = False
correct_letters = []

while not game_over:

    # TODO-6: - Update the code below to tell the user how many lives they have left.
    print(f"****************************** {lives} LIVES LEFT******************************")
    guess = input("Guess a letter: ").lower()

    # TODO-4: - If the user has entered a letter they've already guessed, print the letter and let them know.
    if guess in correct_letters:
        print(f"You've already guessed {guess}")

    display = ""

```

```
36      for letter in chosen_word:
37          if letter == guess:
38              display += letter
39              correct_letters.append(guess)
40          elif letter in correct_letters:
41              display += letter
42          else:
43              display += "_"
44
45      print("Word to guess: " + display)
46
47      # TODO-5: - If the letter is not in the
    chosen_word, print out the letter and let them know
    it's not in the word.
48      #  e.g. You guessed d, that's not in the word.
    You lose a life.
49
50      if guess not in chosen_word:
51          lives -= 1
52          print(f"You guessed {guess} that's not in the
     word. You lose a life")
53          if lives == 0:
54              game_over = True
55
56              # TODO 7: - Update the print statement
    below to give the user the correct word they were
    trying to guess.
57              print(f"********************** IT WAS {
    chosen_word}! YOU LOSE**********************")
58
59      if "_" not in display:
60          game_over = True
61          print("****************************YOU WIN
    ****************************")
62
63      # TODO-2: - Update the code below to use the
    stages List from the file hangman_art.py
64      print(stages[lives])
65
```

(10)

Python
Functions with inputs

Day 8

```
def greet():
    print("Hello")
    print("World")
    print("Bye")
```

greet() -> Deze functie print enkel de strings

```
def greet_with_input(name):
    print(f"Hello {name}")
    print(f"How do you do {name}")
```

greet_with_inputs("Bart") deze functie zal het volgende printen
Hello Bart
How do you do Bart

```
def greet_with(fname, location)
    print(f"Hello {fname}")
    print(f"What is it like in {location}")
```

greet_with("Bart", "Borsbeke") positie is belangrijk

```
def greet_with(fname, location)
    print(f"Hello {fname}")
    print(f"What is it like in {location}")
```

greet_with(fname="Bart", Location="Borsbeke") hier geven we mee
waar de waarde moet komen

11 2 Day 9


```
capitals = {
    "France": "Paris",
    "Germany": "Berlin",
}

travel_log = {
    "France": ["Paris", "Lille", "Dijon"],
    "Germany": ["Stuttgart", "Berlin", "Koblenz", "Mun
}

print(travel_log["France"][1]) # = Lille

nested_list = ["A", "B", ["C", "D"]]

print(nested_list[2][1]) # = D

travel_log = {
  "France": {
    "cities_visited": ["Paris", "Lille", "Dijon"],
    "total_visits": 12
   },
  "Germany": {
    "cities_visited": ["Berlin", "Hamburg", "Stuttgart
    "total_visits": 5
   },
}

print(travel_log["Germany"]["cities_visited"][2]) # =
```

Python

Functions with output, Docstrings, Multiple Return values

Day 10

Functions with output:

Functies met input kunnen we, dat blijft zo maar met return kunnen we nu ook een output krijgen

```
def my_function():
    result = 3 * 2
    return result
output = my_function()
print(output)
```

```
def format_name(f_name, l_name):
    formated_f_name = f_name.title()
    formated_l_name = l_name.title()
    return f"{formated_f_name} {formated_l_name}"

print(format_name("bart", "brondeel"))
```

door .title() te gebruiken zal onze tekst starten met een hoofdletter
het volgende wordt geprint Bart Brondeel

Multiple Return Values:

```
def function1(text):
    return text + text

def function2(text):
    return text.title()
output = function2(function1("hello"))
print(output) = Hellohello
```

Docstrings:

```
def my_function():
"""Met deze 3 aanhalings tekens kunnen we onze functie voorzien
    van tekst op verschillende lynen """
    return 3 * 2
```

(11)

Python

Dictionaries, Nested lists and dictionaries

Day 9

werkt ook zoals een woordenboek, je zoekt een woord en daarnaast staat de definitie

Key: | Value

```
programming_dictionary = {
    "Bug": "An error in a program that prevents the program running as
            expected",
    "Function": "A piece of code that you can easily call over and over again",
    "Loop": "The action of doing something over and over again",
}
```

iets toevoegen

```
programming_dictionary["Bart"] = "the best programmer"
```

oef

```
student_grades = {}
student_scores = {
    "Harry": 88,
    "Ron": 78,
    "Hermione": 95,
    "Draco": 75,
    "Neville": 60,
}
for student in student_scores:
    score = student_score[student]
    if score > 90:
        student_grades[student] = "Outstanding"
    elif score > 80:
        student_grades[student] = "Exceeds expectations"
    elif score>70:
        student_grades[student] = "Acceptable"
    else:
        student_grades[student] = "FAIL"
print(student_grades)
```

(13) Day 11 Blackjack portfolio

File - C:\Users\Bart Sofie\PycharmProjects\100 Days of Code - The Complete Python Pro Bootcamp\Day 11\task\solution.py

```python
import random
from art import logo


def deal_card():
    """Returns a random card from the deck"""
    cards = [11, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 1
    card = random.choice(cards)
    return card


def calculate_score(cards):
    """Take a list of cards and return the score calcu
    if sum(cards) == 21 and len(cards) == 2:
        return 0

    if 11 in cards and sum(cards) > 21:
        cards.remove(11)
        cards.append(1)

    return sum(cards)


def compare(u_score, c_score):
    """Compares the user score u_score against the com
    if u_score == c_score:
        return "Draw 🙃"
    elif c_score == 0:
        return "Lose, opponent has Blackjack 😱"
    elif u_score == 0:
        return "Win with a Blackjack 😎"
    elif u_score > 21:
        return "You went over. You lose 😭"
    elif c_score > 21:
        return "Opponent went over. You win 😁"
    elif u_score > c_score:
        return "You win 😃"
    else:
        return "You lose 😤"


```

```
42 def play_game():
43     print(logo)
44     user_cards = []
45     computer_cards = []
46     computer_score = -1
47     user_score = -1
48     is_game_over = False
49
50     for _ in range(2):
51         user_cards.append(deal_card())
52         computer_cards.append(deal_card())
53
54     while not is_game_over:
55         user_score = calculate_score(user_cards)
56         computer_score = calculate_score(computer_cards
57         print(f"Your cards: {user_cards}, current score
58         print(f"Computer's first card: {computer_cards
59
60         if user_score == 0 or computer_score == 0 or u
61             is_game_over = True
62         else:
63             user_should_deal = input("Type 'y' to get
64             if user_should_deal == "y":
65                 user_cards.append(deal_card())
66             else:
67                 is_game_over = True
68
69     while computer_score != 0 and computer_score < 17:
70         computer_cards.append(deal_card())
71         computer_score = calculate_score(computer_cards
72
73     print(f"Your final hand: {user_cards}, final score
74     print(f"Computer's final hand: {computer_cards}, f
75     print(compare(user_score, computer_score))
76
77
78 while input("Do you want to play a game of Blackjack?
79     print("\n" * 20)
80     play_game()
81
82
```

(14)

Python

Namespaces and scope, Block scope, Global vars, Global contacts

Dag 12

Namespaces and scope:

- variabelen die we definiëren buiten de functie zijn global als we variabelen aanmaken binnen ons functie kunnen we enkel daar aan.
- bv:

```
def my_function():
    naam = "Bart  -> local scope

print(naam) -> naam is niet gekend buiten de functie
```

```
naam = "Bart" -> global scope
def my_function():
    naam = "Sofie"
    print(naam)
my_function()
print(naam) -> Het volgend wordt geprint
Sofie
Bart
```

Python

Describe the problem, Reproduce the bug, Play computer, Fix the errors
Use print, Use a debugger

Day 13

1) Beschrijf het probleem, als het niet duidelijk is probeer dan zoveel mogelijk het probleem in kleine stukje te kappen tot het duidelijk wordt

2 Reproduceer het probleem, test je code meermaals zodat je de errors eruit kan halen die maar af en toe voorkomen zoals bv: in een random functie waarbij men geen rekening houdt met de index die begint bij 0

3 Speel computer: door rustig door de code te lopen (denken) en denken zoals een computer kan je al heel wat bugs voorkomen
kijk ook na wat jezelf verwacht bij de code die je schrijf -> true or false

4 Fix the problems: los de problemen in vs code op (rode lijnen, Error syntax in de console, indien je de error niet begrijpt ben meer leeds googlen, we kunnen ook gebruik maken van try en except Value error:
hoe meer problemen we kunnen oplossen, hoe sterker we worden

5 Gebruik print: om onse code te controleren op de uitkomst die we verwachten kunnen we print gebruiken, is snel en duidelijk
controle op variabelen, waarden, functies ens..

6 gebruik een debugger: er zit al een goede debugger in vs code maar er zijn ook online debuggers
Thonny
pythontutor.com
Deze lopen stap voor stap door de code, je kan ook breakpoints zetten
ook in PyCharm zit een goede debugger

16 Day 14 Portfolio

File - C:\Users\Bart Sofie\PycharmProjects\100 Days of Code - The Complete Python Pro Bootcamp\Day 14\Higher or Lower P

```
# Display art
from art import logo, vs
from game_data import data
import random


def format_data(account):
    """Takes the account data and returns the printable
    account_name = account["name"]
    account_descr = account["description"]
    account_country = account["country"]
    return f"{account_name}, a {account_descr}, from {a


def check_answer(user_guess, a_followers, b_followers)
    """Take a user's guess and the follower counts and
    if a_followers > b_followers:
        return user_guess == "a"
    else:
        return user_guess == "b"


print(logo)
score = 0
game_should_continue = True
# Generate a random account from the game data
account_b = random.choice(data)

# Make the game repeatable.
while game_should_continue:

    # Making account at position B become the next acc
    account_a = account_b
    account_b = random.choice(data)

    if account_a == account_b:
        account_b = random.choice(data)

    print(f"Compare A: {format_data(account_a)}.")
    print(vs)
    print(f"Against B: {format_data(account_b)}.")
```

```
42
43      # Ask user for a guess.
44      guess = input("Who has more followers? Type 'A' or
45
46      # Clear the screen
47      print("\n" * 20)
48      print(logo)
49
50      # - Get follower count of each account
51      a_follower_count = account_a["follower_count"]
52      b_follower_count = account_b["follower_count"]
53
54      # Check if user is correct.
55      is_correct = check_answer(guess, a_follower_count,
56
57      # Give user feedback on their guess.
58      # score keeping.
59      if is_correct:
60          score += 1
61          print(f"You're right! Current score {score}")
62      else:
63          print(f"Sorry, that's wrong. Final score: {sco
64          game_should_continue = False
65
66
67
```

Coffee Machine Program Requirements

1. **Prompt user by asking "`What would you like? (espresso/latte/cappuccino):`"**
    a. Check the user's input to decide what to do next.
    b. The prompt should show every time action has completed, e.g. once the drink is dispensed. The prompt should show again to serve the next customer.

2. **Turn off the Coffee Machine by entering "`off`" to the prompt.**
    a. For maintainers of the coffee machine, they can use "off" as the secret word to turn off the machine. Your code should end execution when this happens.

3. **Print report.**
    a. When the user enters "report" to the prompt, a report should be generated that shows the current resource values. e.g.
    ```
    Water: 100ml
    Milk: 50ml
    Coffee: 76g
    Money: $2.5
    ```

4. **Check resources sufficient?**
    a. When the user chooses a drink, the program should check if there are enough resources to make that drink.
    b. E.g. if Latte requires 200ml water but there is only 100ml left in the machine. It should not continue to make the drink but print: "`Sorry there is not enough water.`"
    c. The same should happen if another resource is depleted, e.g. milk or coffee.

5. **Process coins.**
    a. If there are sufficient resources to make the drink selected, then the program should prompt the user to insert coins.
    b. Remember that quarters = $0.25, dimes = $0.10, nickles = $0.05, pennies = $0.01
    c. Calculate the monetary value of the coins inserted. E.g. 1 quarter, 2 dimes, 1 nickel, 2 pennies = 0.25 + 0.1 x 2 + 0.05 + 0.01 x 2 = $0.52

6. **Check transaction successful?**
    a. Check that the user has inserted enough money to purchase the drink they selected. E.g Latte cost $2.50, but they only inserted $0.52 then after counting the coins the program should say "`Sorry that's not enough money. Money refunded.`".
    b. But if the user has inserted enough money, then the cost of the drink gets added to the machine as the profit and this will be reflected the next time "report" is triggered. E.g.
    ```
    Water: 100ml
    Milk: 50ml
    Coffee: 76g
    Money: $2.5
    ```
    c. If the user has inserted too much money, the machine should offer change.

E.g. "Here is $2.45 dollars in change." The change should be rounded to 2 decimal places.

7. **Make Coffee.**
   a. If the transaction is successful and there are enough resources to make the drink the user selected, then the ingredients to make the drink should be deducted from the coffee machine resources.

      E.g. report before purchasing latte:
      Water: 300ml
      Milk: 200ml
      Coffee: 100g
      Money: $0

      Report after purchasing latte:
      ```
      Water: 100ml
      Milk: 50ml
      Coffee: 76g
      Money: $2.5
      ```

   b. Once all resources have been deducted, tell the user "Here is your latte. Enjoy!". If latte was their choice of drink.

17 Day 15 Portfolio


```python
MENU = {
    "espresso": {
        "ingredients": {
            "water": 50,
            "coffee": 18,
        },
        "cost": 1.5,
    },
    "latte": {
        "ingredients": {
            "water": 200,
            "milk": 150,
            "coffee": 24,
        },
        "cost": 2.5,
    },
    "cappuccino": {
        "ingredients": {
            "water": 250,
            "milk": 100,
            "coffee": 24,
        },
        "cost": 3.0,
    }
}

profit = 0
resources = {
    "water": 300,
    "milk": 200,
    "coffee": 100,
}


def is_resource_sufficient(order_ingredients):
    """Returns True when order can be made, False if i
    for item in order_ingredients:
        if order_ingredients[item] > resources[item]:
            print(f"Sorry there is not enough {item}."
            return False
    return True
```

```
42
43
44 def process_coins():
45     """Returns the total calculated from coins inserte
46     print("Please insert coins.")
47     total = int(input("how many quarters?: ")) * 0.25
48     total += int(input("how many dimes?: ")) * 0.1
49     total += int(input("how many nickles?: ")) * 0.05
50     total += int(input("how many pennies?: ")) * 0.01
51     return total
52
53
54 def is_transaction_successful(money_received, drink_co
55     """Return True when the payment is accepted, or Fa
56     if money_received >= drink_cost:
57         change = round(money_received - drink_cost, 2)
58         print(f"Here is ${change} in change.")
59         global profit
60         profit += drink_cost
61         return True
62     else:
63         print("Sorry that's not enough money. Money re
64         return False
65
66
67 def make_coffee(drink_name, order_ingredients):
68     """Deduct the required ingredients from the resour
69     for item in order_ingredients:
70         resources[item] -= order_ingredients[item]
71     print(f"Here is your {drink_name} ☕. Enjoy!")
72
73
74 is_on = True
75
76 while is_on:
77     choice = input("What would you like? (espresso/lat
78     if choice == "off":
79         is_on = False
80     elif choice == "report":
81         print(f"Water: {resources['water']}ml")
82         print(f"Milk: {resources['milk']}ml")
```

```
83          print(f"Coffee: {resources['coffee']}g")
84          print(f"Money: ${profit}")
85      else:
86          drink = MENU[choice]
87          if is_resource_sufficient(drink["ingredients"]
88              payment = process_coins()
89              if is_transaction_successful(payment, dri
90                  make_coffee(choice, drink["ingredient
91
```

```python
from turtle import Screen
from snake import Snake
from food import Food
from scoreboard import Scoreboard
import time

def start_game():

    screen.clear()
    screen.setup(width=600, height=600)
    screen.bgcolor("black")
    screen.title("Snake Game")
    screen.tracer(0)

    snake = Snake()
    food = Food()
    scoreboard = Scoreboard()

    screen.listen()
    screen.onkey(snake.up, "Up")
    screen.onkey(snake.down, "Down")
    screen.onkey(snake.left, "Left")
    screen.onkey(snake.right, "Right")

    game_is_on = True
    while game_is_on:
        screen.update()
        time.sleep(0.1)
        snake.move()

        # detect collision with food
        if snake.head.distance(food) < 15:
            food.refresh()
            snake.extend()
            scoreboard.increase_score()

        # detect wall collision
        if snake.head.xcor() > 280 or snake.head.xcor(
            game_is_on = False
            scoreboard.game_over()

```

```
42          # detect tail collision
43          for segment in snake.segments[1:]:
44              if snake.head.distance(segment) < 10:
45                  game_is_on = False
46                  scoreboard.game_over()
47
48      # na game over
49      scoreboard.play_again_screen()
50
51      # toetsen binden voor Y/N
52      screen.onkey(lambda: start_game(), "y")
53      screen.onkey(lambda: screen.bye(), "n")
54      screen.listen()
55
56
57  # ----------------------------------
58  # Programma start hier
59  # ----------------------------------
60  screen = Screen()
61  start_game()
62
63  screen.mainloop()
64
```

20 Deel 1 snake Portfolio Snake GAME


```
1 from turtle import Turtle, Screen
2 STARTING_POSITIONS = [(0, 0), (-20, 0), (-40, 0) ]
3 MOVE_DISTANCE = 20
4 UP = 90
5 DOWN = 270
6 LEFT = 180
7 RIGHT = 0
8
9 class Snake:
10
11     def __init__(self):
12         self.segments = []
13         self.create_snake()
14         self.head = self.segments[0]
15
16     def create_snake(self):
17         for positions in STARTING_POSITIONS:
18             self.add_segment(positions)
19
20     def add_segment(self, positions):
21         new_segment = Turtle("square")
22         new_segment.color("white")
23         new_segment.penup()
24         new_segment.goto(positions)
25         self.segments.append(new_segment)
26
27     def extend(self):
28         #add a new segment to the snake
29         self.add_segment(self.segments[-1].position())
30
31     def move(self):
32         for seg_num in range(len(self.segments) - 1, 0
33             new_x = self.segments[seg_num - 1].xcor()
34             new_y = self.segments[seg_num - 1].ycor()
35             self.segments[seg_num].goto(new_x, new_y)
36         self.head.forward(MOVE_DISTANCE)
37
38     def up(self):
39         if self.head.heading() != DOWN:
40             self.head.setheading(UP)
41
```

```
42      def down(self):
43          if self.head.heading() != UP:
44              self.head.setheading(DOWN)
45
46      def left(self):
47          if self.head.heading() != RIGHT:
48              self.head.setheading(LEFT)
49
50      def right(self):
51          if self.head.heading() != LEFT:
52              self.head.setheading(RIGHT)
53
```

(21) scoreboard portfolio Snake GAME


```python
from turtle import Turtle
ALIGNMENT = "center"
FONT = ("Arial", 24, "normal")

class Scoreboard(Turtle):
    def __init__(self):
        super().__init__()
        self.score = 0
        self.color("white")
        self.penup()
        self.goto(0, 270)
        self.hideturtle()
        self.update_scoreboard()

    def update_scoreboard(self):
        self.write(f"Score: {self.score}", align=ALIGN

    def game_over(self):
        self.goto(0.0, 0.0)
        self.write("GAME OVER", align=ALIGNMENT, font=

    def increase_score(self):
        self.score += 1
        self.clear()
        self.update_scoreboard()

    def play_again_screen(self):
        self.goto(0, -30)
        self.write("Play again? (Y/N)", align=ALIGNMEN

```

(21) food Portfolio Snake GAME


```python
from turtle import Turtle
import random

class Food(Turtle):

    def __init__(self):
        super().__init__()
        self.shape("turtle")
        self.penup()
        self.shapesize(stretch_len=0.5, stretch_wid=0.5
        self.color("lime")
        self.speed("slow")
        self.refresh()


    def refresh(self):
        random_x = random.randint(-270, 270)
        random_y = random.randint(-270, 270)
        self.goto(random_x, random_y)

```

(22) Pong GAME main portfolio


```python
from turtle import  Screen
from paddle import Paddle
from ball import Ball
from scoreboard import Scoreboard
import time

screen = Screen()
screen.setup(height=600, width=800)
screen.bgcolor("black")
screen.title("Pong Game")
screen.tracer(0)

r_paddle = Paddle((350, 0))
l_paddle = Paddle((-350, 0))
ball =  Ball(0)
scoreboard = Scoreboard()

screen.listen()
screen.onkey(r_paddle.go_up, "Up")
screen.onkey(r_paddle.go_down, "Down")
screen.onkey(l_paddle.go_up, "z")
screen.onkey(l_paddle.go_down, "s")

game_is_on = True
while game_is_on:
    time.sleep(ball.move_speed)
    screen.update()
    ball.move()

    # Detect collision with the wall
    if ball.ycor() > 280 or ball.ycor() < -280:
        ball.bounce_y()

    # Detect collision with the wall
    if ball.distance(r_paddle) < 50 and ball.xcor() >
        ball.bounce_x()

    #Detect R paddle misses
    if ball.xcor() > 380:
        ball.reset_position()
        scoreboard.l_point()
```

```
42
43      # Detect L paddle misses
44      if ball.xcor() < -380:
45          ball.reset_position()
46          scoreboard.r_point()
47
48  screen.exitonclick()
49
```

22 Pong GAME paddle portfolio

File - C:\Users\Bart Sofie\OneDrive\Programmeren\Odisee\LevensLang_Leren\100_Days_Of_Code\PycharmProjects\100 Days

```
1  from turtle import Turtle
2
3  class Paddle(Turtle):
4
5      def __init__(self, position):
6          super().__init__()
7          self.shape("square")
8          self.color("white")
9          self.shapesize(stretch_wid=5, stretch_len=1)
10         self.penup()
11         self.goto(position)
12
13     def go_up(self):
14         new_y = self.ycor() + 20
15         self.goto(self.xcor(), new_y)
16
17     def go_down(self):
18         new_y = self.ycor() - 20
19         self.goto(self.xcor(), new_y)
20
21
22
```

22 Bny GAME ball portfolio


```python
from turtle import Turtle

class Ball(Turtle):

    def __init__(self, position):
        super().__init__()
        self.shape("circle")
        self.color("white")
        self.penup()
        self.x_move = 10
        self.y_move = 10
        self.move_speed = 0.1


    def move(self):
        new_x = self.xcor() + self.x_move
        new_y = self.ycor() + self.y_move
        self.goto(new_x, new_y)

    def bounce_y(self):
        self.y_move *= -1

    def bounce_x(self):
        self.x_move *= -1
        self.move_speed *= 0.9

    def reset_position(self):
        self.goto(0, 0)
        self.move_speed = 0.1
        self.bounce_x()
```

22 Bny GAME scoreboard portfolio

File - C:\Users\Bart Sofie\OneDrive\Programmeren\Odisee\LevensLang_Leren\100_Days_Of_Code\PycharmProjects\100 Days

```
1 from turtle import Turtle
2
3 ALIGNMENT = "center"
4 FONT = ("Courier", 80, "normal")
5 class Scoreboard(Turtle):
6
7     def __init__(self):
8         super().__init__()
9         self.color("white")
10        self.penup()
11        self.hideturtle()
12        self.l_score = 0
13        self.r_score = 0
14        self.update_scoreboard()
15
16    def update_scoreboard(self):
17        self.clear()
18        self.goto(-100, 200)
19        self.write(self.l_score, align=ALIGNMENT, font=
20        self.goto(100, 200)
21        self.write(self.r_score, align=ALIGNMENT, font=
22
23    def l_point(self):
24        self.l_score += 1
25        self.update_scoreboard()
26
27    def r_point(self):
28        self.r_score += 1
29        self.update_scoreboard()
```

(23) The Turtle Crossing main Portfolio

File - C:\Users\Bart Sofie\OneDrive\Programmeren\Odisee\LevensLang_Leren\100_Days_Of_Code\100 Days of Code - The Cr

```python
import time
import turtle
from turtle import Screen
from player import Player
from car_manager import CarManager
from scoreboard import Scoreboard


def clear_all_turtles():
    """Verwijdert alle turtles van het scherm zonder r
    for t in turtle.turtles():
        t.hideturtle()
        t.clear()


def run_game(screen):
    """Start één speelronde."""
    # Eerst alles van vorige ronde verwijderen
    clear_all_turtles()
    screen.tracer(0)
    screen.bgcolor("white")

    # Nieuwe objecten aanmaken
    player = Player()
    car_manager = CarManager()
    scoreboard = Scoreboard()

    # Controls
    screen.listen()
    screen.onkey(player.go_up, "Up")

    # Game loop
    game_is_on = True
    while game_is_on:
        time.sleep(0.1)
        screen.update()

        car_manager.create_car()
        car_manager.move_cars()

        # Collision
```

```
42          for car in car_manager.all_cars:
43              if car.distance(player) < 20:
44                  game_is_on = False
45                  scoreboard.game_over()
46
47          # Finish line
48          if player.is_at_finish_line():
49              player.go_to_start()
50              car_manager.level_up()
51              scoreboard.increase_level()
52
53      # Game voorbij → vraag opnieuw
54      scoreboard.ask_play_again()
55
56      # Y/N events
57      screen.onkey(lambda: run_game(screen), "y")
58      screen.onkey(lambda: run_game(screen), "Y")
59      screen.onkey(exit_game, "n")
60      screen.onkey(exit_game, "N")
61
62
63  def exit_game():
64      """Sluit het venster."""
65      turtle.bye()
66
67
68  # ---------------- START HIER ----------------
69  screen = Screen()
70  screen.setup(width=600, height=600)
71  screen.tracer(0)
72
73  run_game(screen)
74
75  screen.mainloop()
76
```

23 player


```python
from turtle import Turtle

STARTING_POSITION = (0, -280)
MOVE_DISTANCE = 10
FINISH_LINE_Y = 280


class Player(Turtle):

    def __init__(self):
        super().__init__()
        self.shape("turtle")
        self.penup()
        self.go_to_start()
        self.setheading(90)

    def go_up(self):
        self.forward(MOVE_DISTANCE)

    def go_to_start(self):
        self.goto(STARTING_POSITION)

    def is_at_finish_line(self):
        return self.ycor() > FINISH_LINE_Y

```

(23) car_manager


```
1  from turtle import Turtle
2  import random
3
4  COLORS = ["red", "orange", "yellow", "green", "blue",
5  STARTING_MOVE_DISTANCE = 5
6  MOVE_INCREMENT = 10
7
8
9  class CarManager:
10
11      def __init__(self):
12          self.all_cars = []
13          self.car_speed = STARTING_MOVE_DISTANCE
14
15      def create_car(self):
16          random_chance = random.randint(1, 6)
17          if random_chance == 1:
18              new_car = Turtle("square")
19              new_car.shapesize(stretch_wid=1, stretch_l
20              new_car.penup()
21              new_car.color(random.choice(COLORS))
22              new_car.goto(300, random.randint(-250, 250
23              self.all_cars.append(new_car)
24
25      def move_cars(self):
26          for car in self.all_cars:
27              car.backward(self.car_speed)
28
29      def level_up(self):
30          self.car_speed += MOVE_INCREMENT
31
```

(23) scoreboard


```
1  from turtle import Turtle
2
3  FONT = ("Courier", 24, "normal")
4
5  class Scoreboard(Turtle):
6
7      def __init__(self):
8          super().__init__()
9          self.level = 1
10         self.hideturtle()
11         self.penup()
12         self.goto(-295, 265)
13         self.update_scoreboard()
14
15     def update_scoreboard(self):
16         self.clear()
17         self.write(f"Level: {self.level}", align="left
18
19     def increase_level(self):
20         self.level += 1
21         self.update_scoreboard()
22
23     def game_over(self):
24         self.goto(0, 20)
25         self.write("GAME OVER", align="center", font=F
26
27     def ask_play_again(self):
28         self.goto(0, -40)
29         self.write("Play again? (Y/N)", align="center"
30
```

15) quiz GAME US States GAME


```python
import turtle
import pandas

# Maak een scherm voor het spel
screen = turtle.Screen()
screen.title("U.S. States Game")

# Laad de kaart van de VS als achtergrondvorm
image = "blank_states_img.gif"
screen.addshape(image)
turtle.shape(image)

# Lees de data in en uit het csv-bestand met alle state
data = pandas.read_csv("50_states.csv")

# Maak een lijst met ALLE staatnamen uit de CSV
all_states = data.state.to_list()

# Lijst om bij te houden welke staten de speler al cor
guessed_states = []

# Het spel loopt totdat de speler alle 50 staten heeft
while len(guessed_states) < 50:

    # Vraag de speler om een staat te raden
    # De title toont hoeveel staten er al correct zijn
    answer_state = screen.textinput(
        title=f"{len(guessed_states)}/50 States Correct
        prompt="What's another state's name?"
    ).title()  # Zorgt ervoor dat invoer consistent is

    # Als speler "Exit" intypt, stoppen we en maken we
    if answer_state == "Exit":

        # Maak een lijst van alle staten die NIET gera
        missing_states = []
        for state in all_states:
            if state not in guessed_states:
                missing_states.append(state)

        # Sla deze lijst op in een csv-bestand om late
```

```
42          new_data = pandas.DataFrame(missing_states)
43          new_data.to_csv("states_to_learn.csv")
44          break
45
46      # Als de speler een geldige staat heeft geraden
47      if answer_state in all_states:
48
49          # Voeg de staat toe aan de lijst van correcte
50          guessed_states.append(answer_state)
51
52          # Maak een nieuwe turtle om de staat op de kaa
53          t = turtle.Turtle()
54          t.hideturtle()  # Maak de turtle onzichtbaar
55          t.penup()       # Zorg dat de turtle niet teke
56
57          # Zoek de rij in de CSV die overeenkomt met de
58          state_data = data[data.state == answer_state]
59
60          # Ga naar de x/y positie van de staat op de ka
61          t.goto(state_data.x.item(), state_data.y.item()
62
63          # Schrijf de naam van de staat op de kaart
64          t.write(answer_state)
65
```

```python
#dictionary comprehension

import random

import pandas as pd

namen =  ['Bart', 'Sofie', 'Anna', 'Emiel', 'Lore', 'S
student_scores = {student:random.randint(10,100) for s
#passed_student = {new_key:new_value for (key, value)
passed_student = {student:score for (student, score) i
print(passed_student)

student_dict =  {
    "student": ['Bart', 'Sofie', 'Anna', 'Emiel', 'Lor
    "score": [95, 85, 88, 87, 80, 75, 70, 65, 83],
}

# Door dictionaries lopen:
#for (key, value) in student_dict.items():
    #print(value)

student_data_frame = pd.DataFrame(student_dict)
print(student_data_frame)

# # Door een data frame lopen
# for (key, value) in student_data_frame.items():
#     print(value)

# Door rijen van een data frame lopen
for (index, row) in student_data_frame.iterrows():
    print(row.student)
```

(26) Nato alfabet nog verder uitwerken nazien


```python
# Keyword Method with iterrows()
# {new_key:new_value for (index, row) in df.iterrows()

import pandas

data = pandas.read_csv("nato_phonetic_alphabet.csv")
#TODO 1. Create a dictionary in this format:
phonetic_dict = {row.letter: row.code for (index, row)
print(phonetic_dict)

#TODO 2. Create a list of the phonetic code words from

word = input("Enter a word: ").upper()
output_list = [phonetic_dict[letter] for letter in wor
print(output_list)

```

Program Higher lower project

1) Display art
2) Generate a random account from the game data
3) Format the account data into printable format
4) Ask the user for a guess
5 check if user is correct
   - Get follower count of each account
   - use if statement to check if user is correct
6) Give user feedback on their guess
7) score keeping
8) Make the game repeatable
9) Making account at position B become the next account at position A.

THE APP BREWERY

100 Days of Python Pledge

I Bart Bronsdel *am committed to completing the 100 days of Python challenge.*

I hereby pledge to work for at least an hour on Python programming for 100 days.

I will keep myself on track, even though some days I might feel tired or frustrated.

I will keep myself accountable, even though I have lots of things to do, I will make this a priority in my life.

I will overcome difficulties and achieve my goal.

I will become a Python developer.

I believe in myself.

Signature: Bart Bronsdel

Date Signed: 15/10/2025